

Reinforcement Learning

Dr. Kavita Mohite (Bhosle)

Introduction of deep reinforcement learning,
Markov Decision Process,
basic framework of reinforcement learning,
challenges of reinforcement learning,
Dynamic programming algorithms for reinforcement
learning,
Q Learning and Deep Q-Networks,
Deep Q recurrent networks,
Simple reinforcement learning for Tic-Tac-Toe.

1. Reinforcement learning differs from supervised learning in a way that in supervised learning the training data has the answer key with it so the model is trained with the correct answer itself.
2. whereas in reinforcement learning, there is no answer but the reinforcement agent decides what to do to perform the given task.
3. In the absence of a training dataset, it is bound to learn from its experience.

4. Reinforcement Learning (RL) is the science of decision making. It is about learning the optimal behavior in an environment to obtain maximum reward.
5. In RL, the data is accumulated from machine learning systems that use a trial-and-error method. Data is not part of the input.
6. RL uses algorithms that learn from outcomes and decide which action to take next.
7. After each action, the algorithm receives feedback that helps it determine whether the choice it made was correct, neutral or incorrect.

8. It is a good technique to use for automated systems that have to make a lot of small decisions without human guidance.

9. RL is an autonomous, self-teaching system that essentially learns by trial and error.

10. It performs actions with the aim of maximizing rewards, or in other words, it is learning by doing in order to achieve the best outcomes.

Reinforcement learning –

Input: The input should be an initial state from which the model will start.

Output: There are many possible outputs as there are a variety of solutions to a particular problem.

Training: The training is based upon the input, The model will return a state and the user will decide to reward or punish the model based on its output.

The model keeps continues to learn.

The best solution is decided based on the maximum reward.

Difference between Reinforcement learning and Supervised learning:

Reinforcement learning	Supervised learning
RL is all about making decisions sequentially. In simple words, we can say that the output depends on the state of the current input and the next input depends on the output of the previous input	In Supervised learning, the decision is made on the initial input or the input given at the start
In RL decision is dependent, So we give labels to sequences of dependent decisions	In supervised learning the decisions are independent of each other so labels are given to each decision.
Example: Chess game, text summarization	Example: Object recognition, spam detection

Positive: Positive Reinforcement is defined as when an event, occurs due to a particular behavior, increases the strength and the frequency of the behavior. In other words, it has a positive effect on behavior. Advantages of reinforcement learning are:

- Maximizes Performance

- Sustain Change for a long period of time

- Too much Reinforcement can lead to an overload of states which can diminish the results

Negative: Negative Reinforcement is defined as strengthening of behavior because a negative condition is stopped or avoided.

Advantages of reinforcement learning:

- Increases Behavior

- Provide defiance to a minimum standard of performance

- It Only provides enough to meet up the minimum behavior

Neutral

Elements of Reinforcement Learning

Reinforcement learning elements are as follows:

Policy: Policy defines the learning agent behavior for given time period. It is a mapping from perceived states of the environment to actions to be taken when in those states.

Reward function: Reward function is used to define a goal in a reinforcement learning problem. A reward function is a function that provides a numerical score based on the state of the environment

Value function: Value functions specify what is good in the long run. The value of a state is the total amount of reward an agent can expect to accumulate over the future, starting from that state.

Model of the environment: Models are used for planning.

Various Practical Applications of Reinforcement Learning –

RL can be used in **robotics for industrial automation.**

RL can be used in **machine learning and data processing**

RL can be used to create training systems that provide custom instruction and materials according to the requirement of students.

Application of Reinforcement Learnings

1. **Robotics:** Robots with pre-programmed behavior are useful in structured environments, such as the assembly line of an automobile manufacturing plant, where the task is repetitive in nature.
2. A master **chess player** makes a move. The choice is informed both by planning, anticipating possible replies and counter replies.
3. An adaptive controller adjusts parameters of a **petroleum refinery's operation** in real time.

RL can be used in large environments in the following situations:

A model of the **environment is known**, but an **analytic solution is not available**;

Only a **simulation model** of the environment is given (the subject of simulation-based optimization)

The only way to collect information about the environment is to interact with

Advantages of Reinforcement learning

1. Reinforcement learning can be used to solve very complex problems that cannot be solved by conventional techniques.
2. The model can correct the errors that occurred during the training process.
3. In RL, training data is obtained via the direct interaction of the agent with the environment
4. Reinforcement learning can handle environments that are non-deterministic, meaning that the outcomes of actions are not always predictable. This is useful in real-world applications where the environment may change over time or is uncertain.

Advantages of Reinforcement learning

5. Reinforcement learning can be used to solve a wide range of problems, including those that involve decision making, control, and optimization.

6. Reinforcement learning is a flexible approach that can be combined with other machine learning techniques, such as deep learning, to improve performance.

Disadvantages of Reinforcement learning

1. Reinforcement learning is not preferable to use for solving simple problems.
2. Reinforcement learning needs a lot of computation
3. Reinforcement learning is highly dependent on the quality of the reward function. If the reward function is poorly designed, the agent may not learn the desired behavior.
4. Reinforcement learning can be difficult to debug and interpret. It is not always clear why the agent is behaving in a certain way, which can make it difficult to diagnose and fix problems.

Markov Decision Process

It allows machines and software agents to automatically determine the ideal behavior within a specific context, in order to maximize its performance.

Simple reward feedback is required for the agent to learn its behavior; this is known as the reinforcement signal.

There are many different algorithms that tackle this issue.

In the problem, an agent is supposed to decide the best action to select based on his current state. When this step is repeated, the problem is known as a **Markov Decision Process**.

A **Markov Decision Process (MDP)** model contains:

A set of possible world states S .

A set of Models.

A set of possible actions A .

A real-valued reward function $R(s,a)$.

A policy the solution of **Markov Decision Process**.

States:	S
Model:	$T(S, a, S') \sim P(S' \mid S, a)$
Actions:	$A(S), A$
Reward:	$R(S), R(S, a), R(S, a, S')$
Policy:	$\Pi(S) \rightarrow a$ Π^*

Markov Decision Process

State

A **State** is a set of tokens that represent every state that the agent can be in.

Model

A **Model** (sometimes called Transition Model) gives an action's effect in a state. In particular, $T(S, a, S')$ defines a transition T where being in state S and taking an action 'a' takes us to state S' (S and S' may be the same). For stochastic actions (noisy, non-deterministic) we also define a probability $P(S' | S, a)$ which represents the probability of reaching a state S' if action 'a' is taken in state S . Note Markov property states that the effects of an action taken in a state depend only on that state and not on the prior history.

Actions

An **Action** A is a set of all possible actions. $A(s)$ defines the set of actions that can be taken being in state S .

Reward

A **Reward** is a real-valued reward function. $R(s)$ indicates the reward for simply being in the state S . $R(S, a)$ indicates the reward for being in a state S and taking an action 'a'. $R(S, a, S')$ indicates the reward for being in a state S , taking an action 'a' and ending up in a state S' .

Policy

A **Policy** is a solution to the Markov Decision Process. A policy is a mapping from S to a . It indicates the action 'a' to be taken while in state S .

Q-learning

The agent while being in that state may choose from a set of allowable actions which may fetch different *rewards* (or penalties).

Over time, The learning agent learns to maximize these rewards to behave optimally at any given state it is in.

Q-learning is a basic form of Reinforcement Learning that uses Q-values (also called action values) to iteratively improve the behavior of the learning agent.

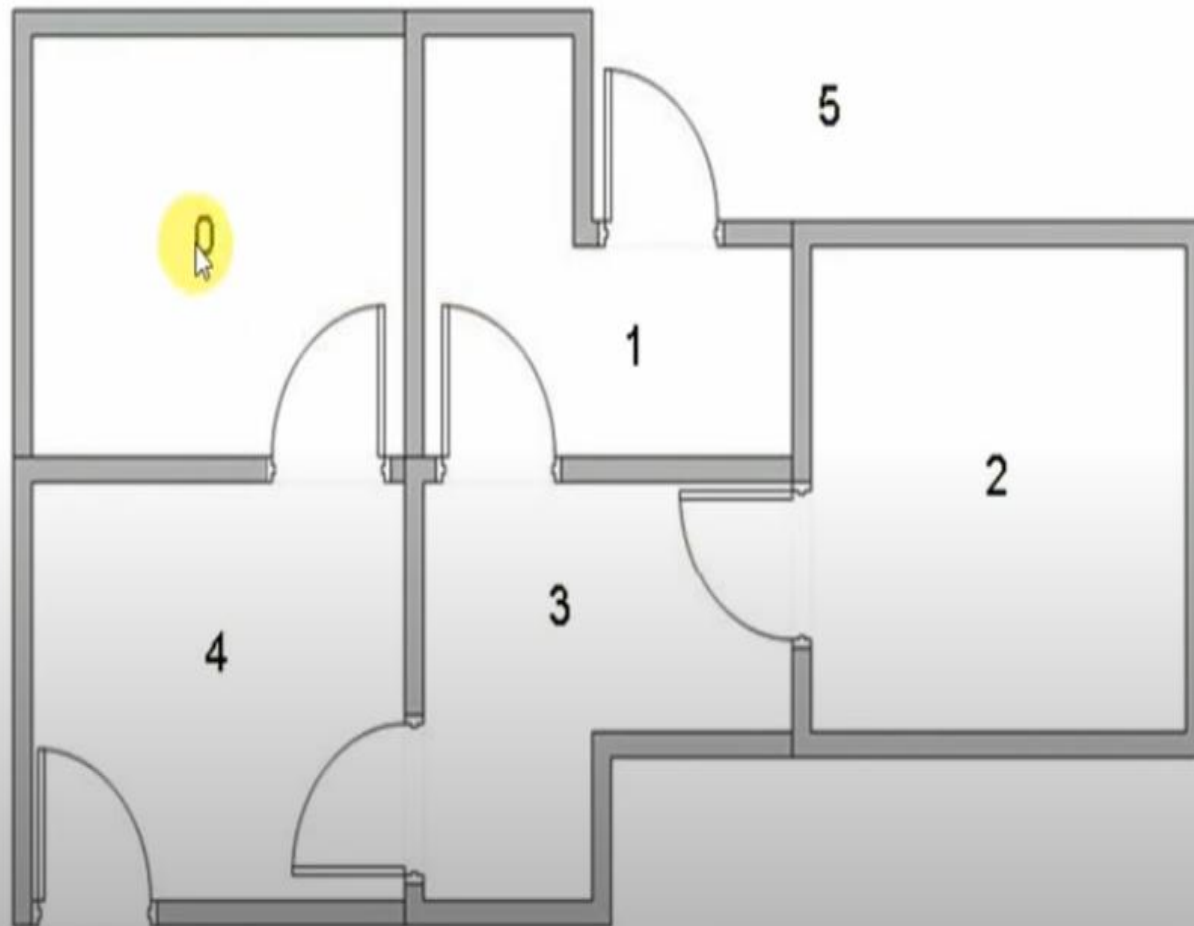
Q-learning in Reinforcement Learning

Q-learning is a popular model-free reinforcement learning algorithm used in machine_learning and artificial intelligence applications. It falls under the category of temporal difference learning techniques, in which an agent picks up new information by observing results, interacting with the environment, and getting feedback in the form of rewards.

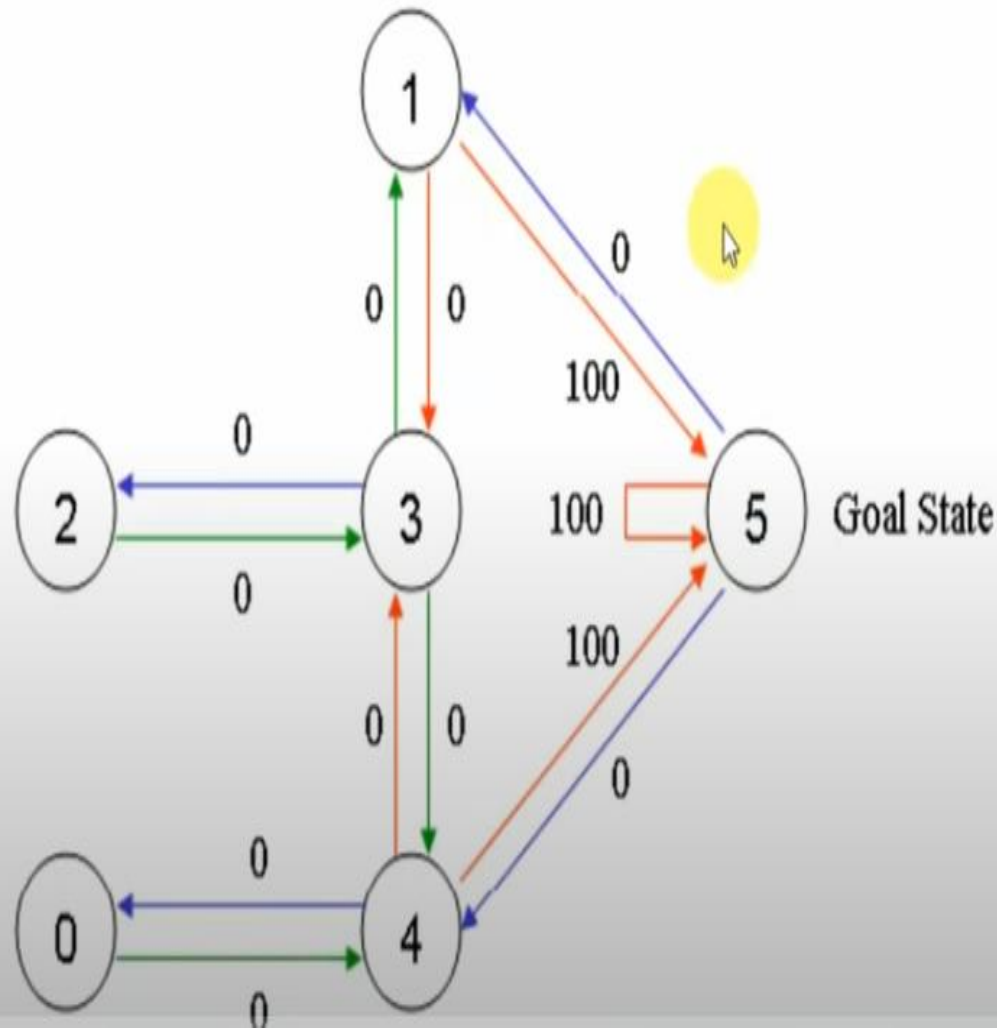
Components of Q-learning

Q-Values or Action-Values: Q-values are defined for states and actions. $Q(S, A)$ is an estimation of how good is it to take the action A at the state S . This estimation of $Q(S, A)$ will be iteratively computed using the **TD- Update rule** which we will see in the upcoming sections.

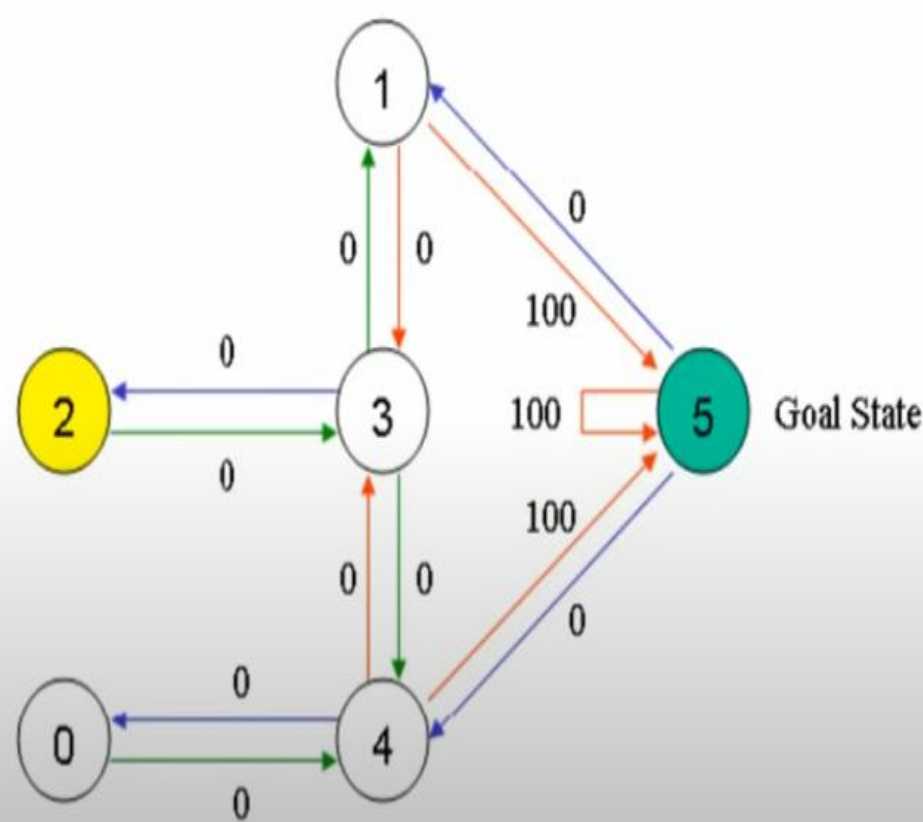
Suppose we have 5 rooms in a building connected by doors as shown in the figure below. We'll number each room 0 through 4. The outside of the building can be thought of as one big room (5). Notice that doors 1 and 4 lead into the building from room 5 (outside).



- The goal room is number 5
- The doors that lead immediately to the goal have an instant reward of 100. Other doors not directly connected to the target room have zero reward.
- Each arrow contains an instant reward value, as shown below:



- We can put the state diagram and the instant reward values into the following reward table, "matrix R". The -1's in the table represent null values (i.e.; where there isn't a link between nodes). For example, State 0 cannot go to State 1.



State Action

	0	1	2	3	4	5
0	-1	-1	-1	-1	0	-1
1	-1	-1	-1	0	-1	100
2	-1	-1	-1	0	-1	-1
3	-1	0	0	-1	0	-1
4	0	-1	-1	0	-1	100
5	-1	0	-1	-1	0	100

$R =$

Q learning algorithm

For each s, a initialize the table entry $\hat{Q}(s, a)$ to zero.

Observe the current state s

Do forever:

- Select an action a and execute it
- Receive immediate reward r
- Observe the new state s'
- Update the table entry for $\hat{Q}(s, a)$ as follows:

$$\hat{Q}(s, a) \leftarrow r + \gamma \max_{a'} \hat{Q}(s', a')$$

- $s \leftarrow s'$

- Now let's imagine what would happen if our agent were in state 5.
- Look at the sixth row of the reward matrix R (i.e. state 5).
- It has 3 possible actions: go to state 1, 4 or 5.
- $Q(\text{state}, \text{action}) = R(\text{state}, \text{action}) + \text{Gamma} * \text{Max}[Q(\text{next state}, \text{all actions})]$
- $Q(1, 5) = R(1, 5) + 0.8 * \text{Max}[Q(5, 1), Q(5, 4), Q(5, 5)] = 100 + 0.8 * 0 = 100$

$$R = \begin{array}{c|cccccc} & \text{Action} \\ \text{State} & 0 & 1 & 2 & 3 & 4 & 5 \\ \hline 0 & -1 & -1 & -1 & -1 & 0 & -1 \\ 1 & -1 & -1 & -1 & 0 & -1 & 100 \\ 2 & -1 & -1 & -1 & 0 & -1 & -1 \\ 3 & -1 & 0 & 0 & -1 & 0 & -1 \\ 4 & 0 & -1 & -1 & 0 & -1 & 100 \\ 5 & -1 & 0 & -1 & -1 & 0 & 100 \end{array}$$

$$Q = \begin{array}{c|cccccc} & 0 & 1 & 2 & 3 & 4 & 5 \\ \hline 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 100 \\ 2 & 0 & 0 & 0 & 0 & 0 & 0 \\ 3 & 0 & 0 & 0 & 0 & 0 & 0 \\ 4 & 0 & 0 & 0 & 0 & 0 & 0 \\ 5 & 0 & 0 & 0 & 0 & 0 & 0 \end{array}$$

- Now we imagine that we are in state 1 (next state).
- Look at the second row of reward matrix R (i.e. state 1).
- It has 2 possible actions: go to state 3 or state 5.
- Then, we compute the Q value:
- $Q(\text{state}, \text{action}) = R(\text{state}, \text{action}) + \text{Gamma} * \text{Max}[Q(\text{next state}, \text{all actions})]$
- $Q(3, 1) = R(3, 1) + 0.8 * \text{Max}[Q(1, 3), Q(1, 5)] = 0 + 0.8 * \text{Max}(0, 100) = 80$

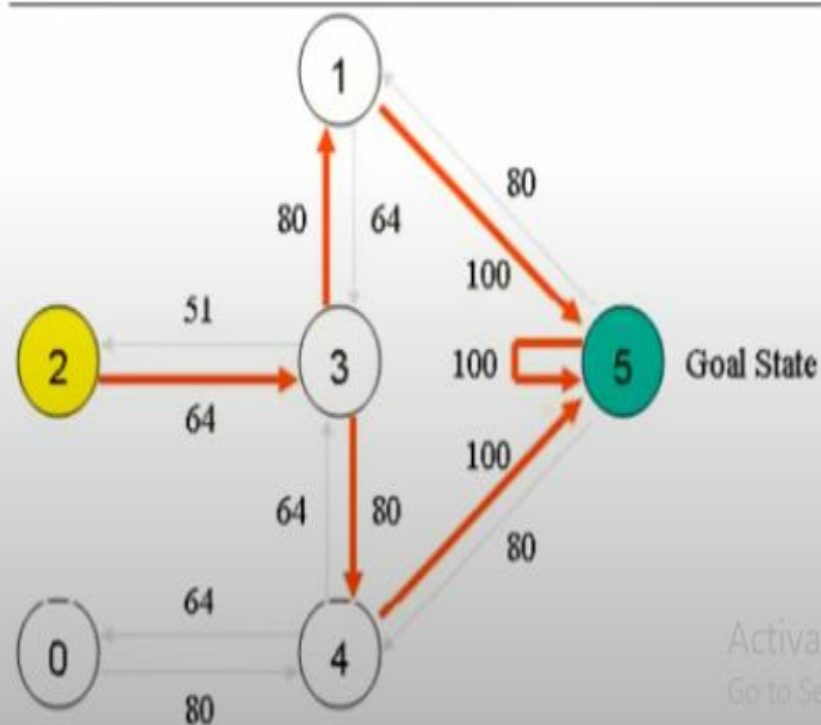
State	Action					
	0	1	2	3	4	5
0	-1	-1	-1	-1	0	-1
1	-1	-1	-1	0	-1	100
2	-1	-1	-1	0	-1	-1
3	-1	0	0	-1	0	-1
4	0	-1	-1	0	-1	100
5	-1	0	-1	-1	0	100

	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	0	0	0	0
2	0	0	0	0	0	100
3	0	80	0	0	0	0
4	0	0	0	0	0	0
5	0	0	0	0	0	0

- If our agent learns more through further episodes, it will finally reach convergence values in matrix Q like:

$$Q = \begin{matrix} & \begin{matrix} 0 & 1 & 2 & 3 & 4 & 5 \end{matrix} \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{matrix} & \begin{bmatrix} 0 & 0 & 0 & 0 & 80 & 0 \\ 0 & 0 & 0 & 64 & 0 & 100 \\ 0 & 0 & 0 & 64 & 0 & 0 \\ 0 & 80 & 51 & 0 & 80 & 0 \\ 64 & 0 & 0 & 64 & 0 & 100 \\ 0 & 80 & 0 & 0 & 80 & 100 \end{bmatrix} \end{matrix}$$

- Tracing the best sequences of states is as simple as following the links with the highest values at each state.

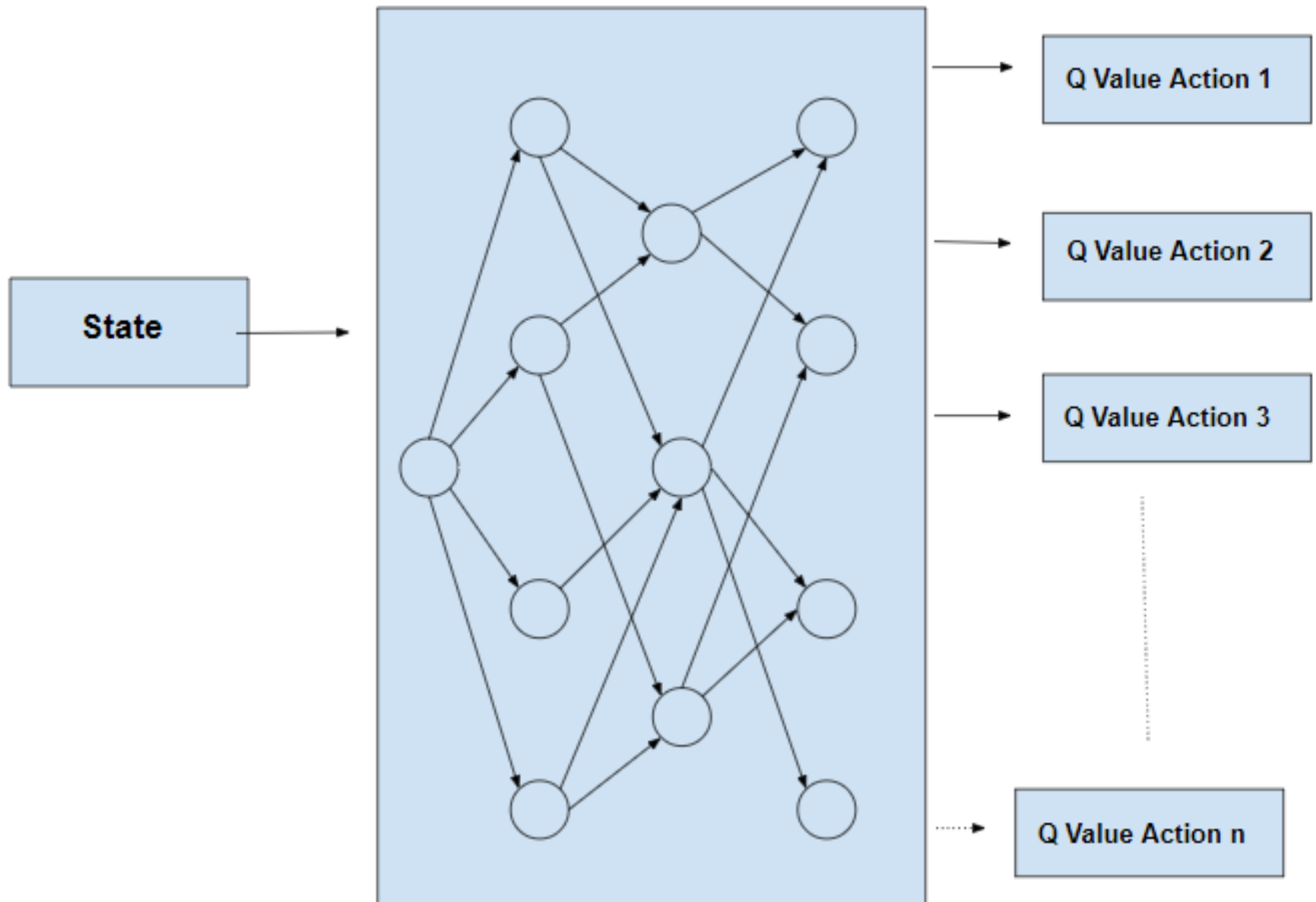


Deep Q-Learning

Q-Learning approach is not wrong in itself, this is only practical for very small environments and quickly loses its feasibility when the number of states and actions in the environment increases. The solution for the above problem comes from the realization that the values in the matrix only have relative importance i.e. the values only have importance with respect to the other values. Thus, this thinking leads us to **Deep Q-Learning** which uses a deep neural network to approximate the values.

The basic working step for Deep Q-Learning is that the initial state is fed into the neural network and it returns the Q-value of all possible actions as an output.

Deep Neural Network



Define the Q-network: The Q-network is a deep neural network that takes in the current state of the agent and outputs the Q-values for each possible action. The Q-network can be defined using TensorFlow's Keras API.

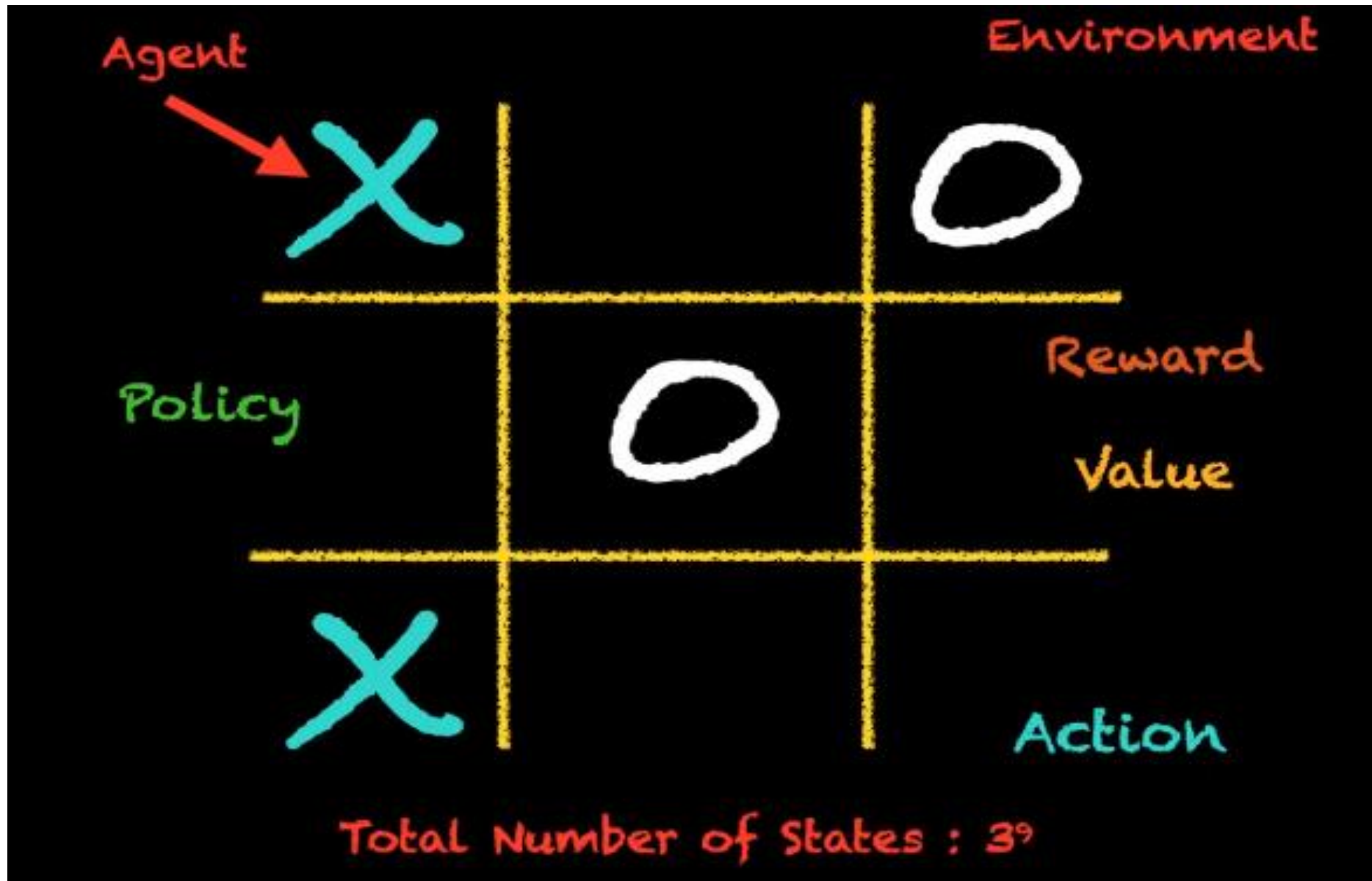
Initialize the Q-network's parameters: The Q-network's parameters can be initialized using TensorFlow's variable initializers.

Define the loss function: The loss function is used to update the Q-network's parameters. The loss function is typically defined as the mean squared error between the Q-network's predicted Q-values and the target Q-values.

Define the optimizer: The optimizer is used to minimize the loss function and update the Q-network's parameters. TensorFlow provides a wide range of optimizers, such as Adam, RMSprop, etc.

Collect experience: The agent interacts with the environment and collects experience in the form of (state, action, reward, next state)

Simple reinforcement learning for Tic-Tac-Toe



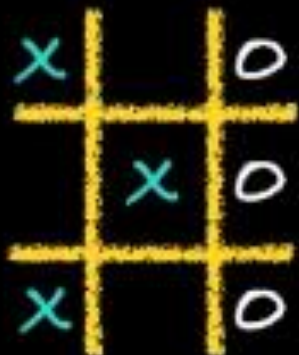
Win!

value = +1



Lose!

value = -1



Draw!

value = 0



Total Number of States : 3^9

